

山东大学 计算机科学与技术 学院

大数据分析实践 课程实验报告

学号：202300130041 姓名： 徐守政 班级： 数据

实验题目：spark 实践

实验学时： 2 实验日期： 2025/11/28

实验目的：
本次 Apache Spark 实践实验，旨在掌握 Spark 的安装配置流程（含环境变量设置、集群搭建与启动）及运行的软硬件要求，通过编写运行简单应用程序熟悉 SparkSession 初始化、数据集加载等基础操作与核心概念，进而提升运用 Spark 执行多数据集连接、Spark SQL 查询等复杂任务，解决销售数据、客流数据等实际业务分析问题的实践能力。

硬件环境：
计算机一台

软件环境：
Windows

```

+-----+-----+-----+-----+-----+-----+
| NVIDIA-SMI 570.133.07 | Driver Version: 570.133.07 | CUDA Version: 12.8 |
+-----+-----+-----+-----+-----+-----+
| GPU  Name                Persistence-M | Bus-Id        Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |      Memory-Usage | GPU-Util  Compute M. |
|====+=====+=====+=====+=====+=====+
|  0  NVIDIA GeForce RTX 3050      Off | 00000000:05:00:00 Off |          0%      N/A |
| 30%   32C    P8              8W / 70W |  37MiB /  6144MiB |             Default  |
+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+
| Processes: |
| GPU   GI    CI          PID    Type   Process name                  GPU Memory |
|====+=====+=====+=====+=====+=====+
|  0     N/A   N/A         2749      G   /usr/bin/gnome-shell           1MiB |
|  0     N/A   N/A         8118     C+G   /usr/bin/ptxix                 3MiB |
|  0     N/A   N/A        245460     C+G   /usr/bin/gnome-control-center  3MiB |
+-----+-----+-----+-----+-----+-----+

```

实验步骤与内容：
上传文件：

```

PS C:\Users\Lenovo> scp D:\Desktop\data\sales_data.csv xushouzheng@10.102.35.53:/home/xushouzheng/
xushouzheng@10.102.35.53's password:
sales_data.csv                                100% 14MB  9.8MB/s   00:01
PS C:\Users\Lenovo> scp D:\Desktop\data\spark-experiment.py xushouzheng@10.102.35.53:/home/xushouzheng/
xushouzheng@10.102.35.53's password:
spark-experiment.py                          100% 6651  590.5KB/s  00:00
PS C:\Users\Lenovo> |

```

将文件导入 HDFS 文件系统中：

```

WARNING! The remote SSH server rejected X11 forwarding request.
Last login: Wed Nov 26 21:04:06 2025 from 172.25.205.78
(base) [xushouzheng@hadoop-master ~]$ hdfs dfs -mkdir -p /user/xushouzheng/spark_data/
(base) [xushouzheng@hadoop-master ~]$ hdfs dfs -put -f /home/xushouzheng/sales_data.csv /user/xushouzheng/spark_data/
2025-11-28 15:19:24,253 INFO ssl.SslDataTransferClient: SASL encryption trust check: localHostTrusted = false, remoteHostTrusted = false
(base) [xushouzheng@hadoop-master ~]$ hdfs dfs -ls /user/xushouzheng/spark_data/
Found 1 items
-rw-r--r--  3 xushouzheng xushouzheng  15121980 2025-11-28 15:19 /user/xushouzheng/spark_data/sales_data.csv
(base) [xushouzheng@hadoop-master ~]$

```

spark 运行:

```
25/11/28 15:21:42 INFO Client: Verifying our application has not requested more than the maximum memory capability of the cluster (8192 MB per container)
25/11/28 15:21:42 INFO Client: Will allocate AM container, with 1408 MB memory including 384 MB overhead
25/11/28 15:21:42 INFO Client: Setting up container launch context for our AM
25/11/28 15:21:42 INFO Client: Setting up the launch environment for our AM container
25/11/28 15:21:42 INFO Client: Preparing resources for our AM container
25/11/28 15:21:42 WARN Client: Neither spark.yarn.jars nor spark.yarn.archive is set, falling back to uploading libraries under SPARK_HOME.
25/11/28 15:21:43 INFO Client: Uploading resource file:/tmp/spark-27573b47-d720-4ea7-b456-39e1071b435d/__spark_libs__1445613839746582404.zip -> hdfs://hdmap-master:8020/user/xushouzheng/.sparkStaging/application_1763789559014_0034/
4/ __spark_libs__1445613839746582404.zip
25/11/28 15:21:46 INFO Client: Uploading resource file:/home/xushouzheng/spark-experiment.py -> hdfs://hdmap-master:8020/user/xushouzheng/.sparkStaging/application_1763789559014_0034/spark-experiment.py
25/11/28 15:21:46 INFO Client: Uploading resource file:/opt/module/spark-yarn/python/lib/pyspark.zip -> hdfs://hdmap-master:8020/user/xushouzheng/.sparkStaging/application_1763789559014_0034/pyspark.zip
25/11/28 15:21:46 INFO Client: Uploading resource file:/opt/module/spark-yarn/python/lib/py4j-0.10.9.5-src.zip -> hdfs://hdmap-master:8020/user/xushouzheng/.sparkStaging/application_1763789559014_0034/py4j-0.10.9.5-src.zip
25/11/28 15:21:46 INFO Client: Uploading resource file:/tmp/spark-27573b47-d720-4ea7-b456-39e1071b435d/__spark_conf__614055596782906027.zip -> hdfs://hdmap-master:8020/user/xushouzheng/.sparkStaging/application_1763789559014_0034/
4/ __spark_conf__614055596782906027.zip
25/11/28 15:21:47 INFO SecurityManager: Changing view acls to: xushouzheng
25/11/28 15:21:47 INFO SecurityManager: Changing modify acls to: xushouzheng
25/11/28 15:21:47 INFO SecurityManager: Changing view acls groups to:
25/11/28 15:21:47 INFO SecurityManager: Changing modify acls groups to:
25/11/28 15:21:47 INFO SecurityManager: SecurityManager: authentication disabled; ui acls disabled; users with view permissions: Set(xushouzheng); groups with view permissions: Set(); users with modify permissions: Set(xushouzheng); groups with modify permissions: Set()
25/11/28 15:21:47 INFO Client: Submitting application application_1763789559014_0034 to ResourceManager
25/11/28 15:21:47 INFO YarnClientImpl: Submitted application application_1763789559014_0034
25/11/28 15:21:48 INFO Client: Application report for application_1763789559014_0034 (state: ACCEPTED)
25/11/28 15:21:48 INFO Client:
  client token: N/A
  diagnostics: AM container is launched, waiting for AM container to Register with RM
  ApplicationMaster host: N/A
  ApplicationMaster RPC port: -1
  queue: default
  start time: 1764314507076
  final status: UNDEFINED
  tracking URL: http://hdmap-master:8088/proxy/application_1763789559014_0034/
  user: xushouzheng
25/11/28 15:21:49 INFO Client: Application report for application_1763789559014_0034 (state: ACCEPTED)
25/11/28 15:21:50 INFO Client: Application report for application_1763789559014_0034 (state: ACCEPTED)
25/11/28 15:21:51 INFO Client: Application report for application_1763789559014_0034 (state: ACCEPTED)
25/11/28 15:21:52 INFO Client: Application report for application_1763789559014_0034 (state: ACCEPTED)
25/11/28 15:21:53 INFO Client: Application report for application_1763789559014_0034 (state: ACCEPTED)
25/11/28 15:21:54 INFO Client: Application report for application_1763789559014_0034 (state: RUNNING)
25/11/28 15:21:54 INFO Client:
  client token: N/A
  diagnostics: N/A
  ApplicationMaster host: hdmap-master
  ApplicationMaster RPC port: 46219
  queue: default
  start time: 1764314507076
  final status: UNDEFINED
  tracking URL: http://hdmap-master:8088/proxy/application_1763789559014_0034/
  user: xushouzheng
25/11/28 15:21:55 INFO Client: Application report for application_1763789559014_0034 (state: RUNNING)
25/11/28 15:21:56 INFO Client: Application report for application_1763789559014_0034 (state: RUNNING)
25/11/28 15:21:57 INFO Client: Application report for application_1763789559014_0034 (state: RUNNING)
25/11/28 15:21:58 INFO Client: Application report for application_1763789559014_0034 (state: RUNNING)
25/11/28 15:21:59 INFO Client: Application report for application_1763789559014_0034 (state: RUNNING)
25/11/28 15:22:00 INFO Client: Application report for application_1763789559014_0034 (state: RUNNING)
25/11/28 15:22:01 INFO Client: Application report for application_1763789559014_0034 (state: RUNNING)
25/11/28 15:22:02 INFO Client: Application report for application_1763789559014_0034 (state: RUNNING)
25/11/28 15:22:03 INFO Client: Application report for application_1763789559014_0034 (state: RUNNING)
25/11/28 15:22:04 INFO Client: Application report for application_1763789559014_0034 (state: RUNNING)
25/11/28 15:22:05 INFO Client: Application report for application_1763789559014_0034 (state: RUNNING)
25/11/28 15:22:06 INFO Client: Application report for application_1763789559014_0034 (state: RUNNING)
25/11/28 15:22:07 INFO Client: Application report for application_1763789559014_0034 (state: RUNNING)
25/11/28 15:22:08 INFO Client: Application report for application_1763789559014_0034 (state: RUNNING)
25/11/28 15:22:09 INFO Client: Application report for application_1763789559014_0034 (state: RUNNING)
25/11/28 15:22:10 INFO Client: Application report for application_1763789559014_0034 (state: RUNNING)
25/11/28 15:22:11 INFO Client: Application report for application_1763789559014_0034 (state: RUNNING)
25/11/28 15:22:12 INFO Client: Application report for application_1763789559014_0034 (state: RUNNING)
25/11/28 15:22:13 INFO Client: Application report for application_1763789559014_0034 (state: FINISHED)
25/11/28 15:22:13 INFO Client:
  client token: N/A
  diagnostics: N/A
  ApplicationMaster host: hdmap-master
  ApplicationMaster RPC port: 46219
  queue: default
  start time: 1764314507076
  final status: SUCCEEDED
  tracking URL: http://hdmap-master:8088/proxy/application_1763789559014_0034/
  user: xushouzheng
25/11/28 15:22:13 INFO ShutdownHookManager: Shutdown hook called
25/11/28 15:22:13 INFO ShutdownHookManager: Deleting directory /tmp/spark-27573b47-d720-4ea7-b456-39e1071b435d
25/11/28 15:22:13 INFO ShutdownHookManager: Deleting directory /tmp/spark-57c55df9-698e-488b-bc7f-76bc5563970f
(base) [xushouzheng@hdmap-master ~]$ []
```

读取日志，获得运行结果:

```
(base) [xushouzheng@hdmap-master ~]$ yarn logs -applicationId application_1763789559014_0034
2025-11-28 15:23:46,743 INFO client.RMProxy: Connecting to ResourceManager at hdmap-master/10.102.35.53:8032
2025-11-28 15:23:47,488 INFO sasl.SaslDataTransferClient: SASL encryption trust check: localHostTrusted = false, remoteHostTrusted = false
2025-11-28 15:23:47,562 INFO sasl.SaslDataTransferClient: SASL encryption trust check: localHostTrusted = false, remoteHostTrusted = false
2025-11-28 15:23:47,572 INFO sasl.SaslDataTransferClient: SASL encryption trust check: localHostTrusted = false, remoteHostTrusted = false
Container: container_1763789559014_0034_01_000001 on hdmap-master_35523
LogAggregationType: AGGREGATED
=====
```

运行结果展示:

```
=====
前置步骤：加载并预处理销售数据
=====
数据预处理完成，新增字段：store_id(虚拟门店ID)、is_weekday(是否工作日)、traffic(客流=订单量)
+-----+-----+-----+-----+
|store_id|order_date|is_weekday|traffic|
+-----+-----+-----+-----+
|British Columbia_Canada|2013-11-26|1|8|
|British Columbia_Canada|2015-11-26|1|8|
|New South Wales_Australia|2014-03-23|0|23|
|New South Wales_Australia|2016-03-23|1|20|
|New South Wales_Australia|2014-05-15|1|4|
+-----+-----+-----+-----+
only showing top 5 rows
```

7.5.1 WordCount词频统计结果（基于产品名称分词）

Word	Count
Tire	23454
Mountain	19108
-	18558
Bottle	15878
Tube	14694
Helmet,	12160
Sport-100	12160
oz.	12126
Road	10904
Water	10794
30	10794
Patch	10416
Kit/8	10416
Patches	10416
Black,	8520
Logo	7580
Mountain-200	6144
Jersey,	6010
Yellow,	5390
Cage	5084

7.5.2 工作日流量最大的门店ID（客流=订单量）

Store_ID	Total_Weekday_Traffic
California_United States	191174
British Columbia_Canada	137485
England_United Kingdom	112637
Washington_United States	100059
New South Wales_Australia	81750
Oregon_United States	47347
Victoria_Australia	46523
Queensland_Australia	41560
Saarland_Germany	22387
Nordrhein-Westfalen_Germany	20190
Seine (Paris)_France	19656
Hessen_Germany	18877
Hamburg_Germany	14403
Seine Saint Denis_France	14265
Nord_France	13728
South Australia_Australia	11211
Bayern_Germany	10811
Hauts de Seine_France	9165
Essonne_France	8751
Yveline_France	7746

only showing top 20 rows

可以看到，工作日客流最大的门店ID为【California_United States】，工作日总客流（订单量）为191174。

7.5.3 每个门店的日均客流量（客流=订单量）

Store_ID	Avg_Traffic	Days	Total_Traffic
Alabama_United States	26.25	4	105
Mississippi_United States	22.75	4	91
Minnesota_United States	21.83	6	131
Ohio_United States	20.43	28	572
Montana_United States	20.33	6	122
Ontario_Canada	19.33	6	116
North Carolina_United States	15.75	4	63
Virginia_United States	15.75	4	63
Georgia_United States	14.0	8	112
New York_United States	13.8	20	276
British Columbia_Canada	13.57	14116	191556
Pas de Calais_France	13.42	90	1208
Val de Marne_France	13.37	158	2112
Kentucky_United States	13.0	10	130
Oregon_United States	12.68	5286	67012
Utah_United States	12.6	10	126
Val d'Oise_France	12.53	264	3307
Somme_France	12.37	134	1657
Loir et Cher_France	12.31	120	1477
Washington_United States	12.25	11264	137947

only showing top 20 rows

代码如下：

```
# 导入PySpark 核心模块

from pyspark.sql import SparkSession

from pyspark.sql.functions import (
```



```
col, sum, avg, desc, concat, lit, dayofweek, count, round, to_date,
explode, split, count as spark_count
)
from pyspark.sql.types import IntegerType
```

```
# 初始化 SparkSession (实验入口)
```

```
spark = SparkSession.builder \
    .appName("Spark_Experiment_OnlySalesData") \
    .master("yarn") \
    .config("spark.serializer", "org.apache.spark.serializer.KryoSerializer") \
    .config("spark.sql.legacy.timeParserPolicy", "LEGACY") \
    .getOrCreate()
```

```
# 关闭冗余日志, 让输出更简洁
```

```
spark.sparkContext.setLogLevel("WARN")
```

```
try:
```

```
# =====
```

```
# 前置步骤: 加载 sales_data.csv 并预处理
```

```
# =====
```

```
print("=*60)
```

```
print("前置步骤: 加载并预处理销售数据")
```

```
print("=*60)
```

```
# 加载 HDFS 中的数据
```

```
sales_df = spark.read \
    .option("header", "true") \
    .option("inferSchema", "true") \
    .csv("hdfs://hdmap-master:8020/user/xushouzheng/spark_data/sales_data.csv")
```

```
# 数据预处理:
```

```
# 1. 拼接虚拟门店 ID (State+Country, 模拟门店维度)
```

```
sales_df = sales_df.withColumn("store_id", concat(col("State"), lit("_"), col("Country")))
```

```
# 2. 日期转换: 兼容单/双位数的年/月/日格式 (yyyy/M/d), 同时结合 LEGACY 策略双重保障
```

```
sales_df = sales_df.withColumn("order_date", to_date(col("Date"), "yyyy/M/d"))
```

```
# 3. 修复工作日判断: dayofweek 返回 1(周日)-7(周六), 2-6 为工作日 (周一到周五)
```

```
sales_df = sales_df.withColumn(
    "is_weekday",
    ((dayofweek(col("order_date")) >= 2) & (dayofweek(col("order_date")) <= 6)).cast(IntegerType())
)
```

```
# 4. 用 Order_Quantity 代表客流量 (traffic)
```

```
sales_df = sales_df.withColumn("traffic", col("Order_Quantity").cast(IntegerType()))
```

```
# 显示预处理结果 (仅前 5 行)
```

```
print("数据预处理完成, 新增字段: store_id(虚拟门店 ID)、is_weekday(是否工作日)、traffic(客流=订单量)")
```

```
sales_df.select("store_id", "order_date", "is_weekday", "traffic").show(5, truncate=False)
```

```
# =====

# 7.5.1 实验: WordCount 词频统计 (DataFrame 实现, 避免序列化问题)

# =====

print("\n" + "="*60)

print("7.5.1 WordCount 词频统计结果 (基于产品名称分词)")

print("="*60)

# 拆分 Product 字段为单词, 生成新行

word_df = sales_df.select(
    explode(split(col("Product"), " ")).alias("word")
).filter(col("word") != "") # 过滤空单词

# 按单词分组计数

word_count_df = word_df.groupBy("word") \
    .agg(spark_count("*").alias("count")) \
    .orderBy(desc("count"))

# 格式化输出 (仅前 20 行)

print("+ " + "-"*15 + "+" + "-"*10 + "+")

print("|      Word      | Count |")

print("+ " + "-"*15 + "+" + "-"*10 + "+")

for row in word_count_df.limit(20).collect():
    print(f"| {row['word'].ljust(13)} | {str(row['count']).ljust(8)} |")

print("+ " + "-"*15 + "+" + "-"*10 + "+")
```

```
# =====

# 7.5.2 实验: 求工作日流量最大的门店 ID

# =====

print("\n" + "="*60)

print("7.5.2 工作日流量最大的门店 ID (客流=订单量)")

print("="*60)

# 过滤掉日期转换失败的行

valid_sales_df = sales_df.filter(col("is_weekday").isNotNull())
```

```
# 按门店分组统计工作日总客流

weekday_traffic = valid_sales_df.filter(col("is_weekday") == 1) \
    .groupBy("store_id") \
    .agg(sum("traffic").alias("total_weekday_traffic")) \
    .orderBy(desc("total_weekday_traffic"))
```

```
# 格式化输出 (仅前 20 行)

print("+ " + "-"*40 + "+" + "-"*20 + "+")

print("|          Store_ID          | Total_Weekday_Traffic |")

print("+ " + "-"*40 + "+" + "-"*20 + "+")

for idx, row in enumerate(weekday_traffic.limit(20).collect()):
    store = row["store_id"].ljust(38)
    traffic = str(row["total_weekday_traffic"]).ljust(18)
    print(f"| {store} | {traffic} |")

print("+ " + "-"*40 + "+" + "-"*20 + "+")

print("only showing top 20 rows")
```

```

# 提取最大流量门店

if weekday_traffic.count() > 0:
    max_store = weekday_traffic.first()

    print(f"\n 可以看到, 工作日客流最大的门店 ID 为【{max_store['store_id']}】, 工作日总客流(订单量)为{max_store['total_weekday_traffic']}。")
)

else:
    print("\n 暂无工作日流量数据!")

# =====

# 7.5.3 实验: 求每个门店的日均客流量

# =====

print("\n" + "="*60)

print("7.5.3 每个门店的日均客流量 (客流=订单量)")

print("="*60)

# 按门店分组计算日均客流

daily_avg_traffic = valid_sales_df.groupBy("store_id") \
    .agg(
        round(avg("traffic"), 2).alias("daily_average_traffic"),
        count("order_date").alias("total_days"),
        sum("traffic").alias("total_traffic")
    ) \
    .orderBy(desc("daily_average_traffic"))

# 格式化输出 (仅前 20 行)

print("+" + "-"*40 + "+" + "-"*15 + "+" + "-"*10 + "+" + "-"*15 + "+")

print("|          Store_ID          | Avg_Traffic | Days | Total_Traffic |")

print("+" + "-"*40 + "+" + "-"*15 + "+" + "-"*10 + "+" + "-"*15 + "+")

for idx, row in enumerate(daily_avg_traffic.limit(20).collect()):
    store = row["store_id"].ljust(38)
    avg_t = str(row["daily_average_traffic"]).ljust(13)
    days = str(row["total_days"]).ljust(8)
    total_t = str(row["total_traffic"]).ljust(13)

    print(f"| {store} | {avg_t} | {days} | {total_t} |")

print("+" + "-"*40 + "+" + "-"*15 + "+" + "-"*10 + "+" + "-"*15 + "+")

print("only showing top 20 rows")

except Exception as e:

    print(f"\n 程序执行出错: {str(e)}")

    # 注释掉raise, 避免集群模式下任务直接崩溃 (可选)

    # raise e

finally:

    # 关闭SparkSession, 释放资源

    spark.stop()

    print("\n" + "="*60)

    print("实验完成, Spark 资源已释放!")

    print("="*60)

```

结论分析与体会：

本次 Spark 实验对销售数据集开展加载、预处理与多维度分析，形成明确业务结论：词频统计环节，通过产品名称分词可直观识别高频产品相关词汇，能够反映热销产品的名称特征；工作日流量分析显示，某一由州和国家拼接而成的虚拟门店 ID，其总工作日客流（以订单量计）显著高于其他门店，体现出该门店在工作日的经营热度与客流吸引力；各门店日均客流量对比中，不同门店间差异明显，部分门店因日均订单量更高，呈现出更优的运营效率。

从实验实践来看，本次操作实现了对 SparkSession 初始化配置、DataFrame 预处理（含日期修复、虚拟字段生成、工作日判断）及核心算子（如 explode、split、groupBy、agg）使用方法的掌握；同时验证了 Spark 在大数据处理中分布式计算的高效性，明确了数据预处理（包括日期格式兼容、空值过滤）对保障分析结果准确性的关键作用；此外，通过 YARN 集群模式部署并配置序列化策略，进一步明晰了 Spark 集群资源调度与性能优化的基础思路，为后续应对更复杂的大数据业务场景提供了实操支撑。